

4 · Parameter estimation & inverse problems ⇌

A very common SciML task: given noisy experimental observations and a mechanistic ODE model with *unknown* parameters, find the parameter values that best explain the data. This is the classic **inverse problem**, solved here by turning it into an optimization problem: minimize the mismatch between simulated and observed trajectories over the ODE parameters.

```
1 using OrdinaryDiffEq, Optimization, OptimizationOptimJL, Plots, Random, PlutoUI
```

1. The model: a SIR epidemic ⇌

$$\begin{aligned}\dot{S} &= -\beta SI, & \dot{I} &= \beta SI - \gamma I, & \dot{R} &= \gamma I \\ & & \beta & & & \end{aligned}$$

is the infection rate and γ the recovery rate. In a real setting these come from data fitting rather than being known constants.

sir! (generic function with 1 method)

```
1 function sir!(du, u, p, t)
2     S, I, R = u
3     β, γ = p
4     du[1] = -β * S * I
5     du[2] = β * S * I - γ * I
6     du[3] = γ * I
7     return nothing
8 end
```

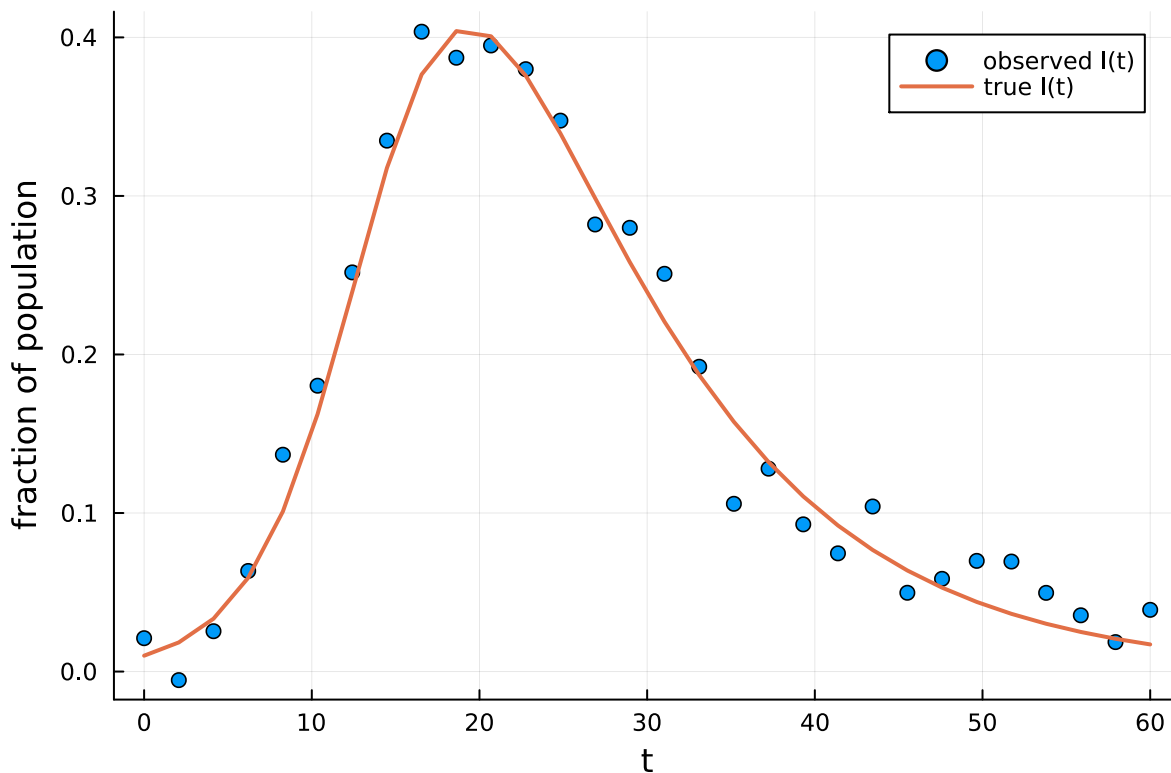
2. Generate noisy "observed" data ⇌

We simulate with known true parameters, then add Gaussian observation noise — mimicking real measurement error.

```
3×30 Matrix{Float64}:
```

```
 0.964679  0.982695  1.00588  0.891249  ...  0.0384385  0.028595  0.00652742
 0.0210441 -0.00540344  0.0254456  0.0634464  ...  0.0354832  0.0185801  0.0389212
-0.014492  0.0200908 -0.0242095 -0.0101834  ...  0.954567  0.957536  0.950197
```

```
1 begin
2   rng4 = Random.default_rng()
3   Random.seed!(rng4, 7)
4
5   true_β, true_γ = 0.4, 0.1
6   u0_sir = [0.99, 0.01, 0.0]
7   tspan_sir = (0.0, 60.0)
8   tsteps_sir = range(tspan_sir[1], tspan_sir[2]; length=30)
9
10  sir_prob = ODEProblem(sir!, u0_sir, tspan_sir, [true_β, true_γ])
11  sir_sol = solve(sir_prob, Tsit5(); saveat=tsteps_sir)
12
13  noise_level = 0.02
14  sir_data = Array(sir_sol) .+ noise_level .* randn(rng4, size(Array(sir_sol)))
15 end
```



```
1 begin
2   scatter(tsteps_sir, sir_data[2, :]; label="observed I(t)", xlabel="t",
3   ylabel="fraction of population")
4   plot!(tsteps_sir, Array(sir_sol)[2, :]; label="true I(t)", lw=2)
5 end
```

3. Loss landscape ⇌

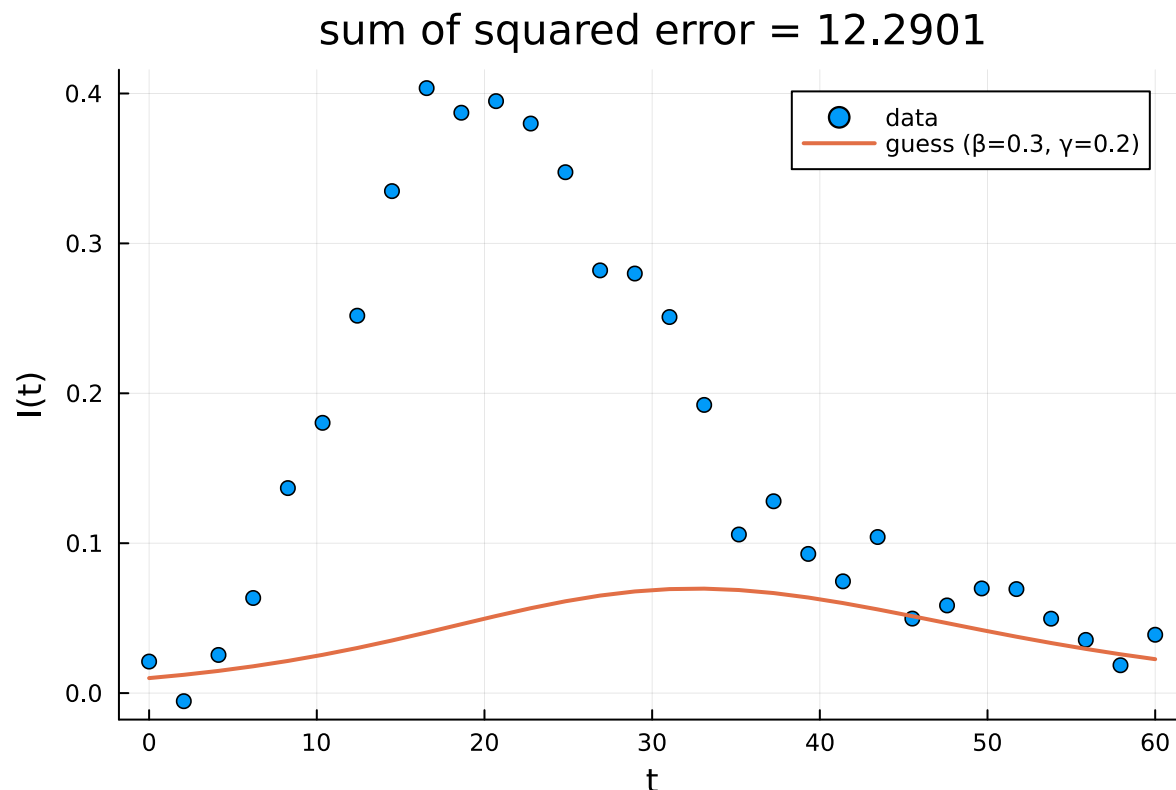
Before fitting, it helps to see how sensitive the mismatch is to each parameter. Drag the sliders below and watch the simulated curve chase the noisy data.

 0.3

```
1 @bind β_guess Slider(0.1:0.02:0.8, default=0.3, show_value=true)
```

 0.2

```
1 @bind γ_guess Slider(0.02:0.02:0.5, default=0.2, show_value=true)
```



```
1 begin
2   guess_prob = remake(sir_prob; p=[β_guess, γ_guess])
3   guess_sol = solve(guess_prob, Tsit5(); saveat=tsteps_sir)
4   guess_loss = sum(abs2, sir_data .- Array(guess_sol))
5
6   scatter(tsteps_sir, sir_data[2, :]; label="data", xlabel="t", ylabel="I(t)")
7   plot!(tsteps_sir, Array(guess_sol)[2, :]; label="guess (β=$(β_guess),
8     γ=$(γ_guess))", lw=2,
9     title="sum of squared error = $(round(guess_loss, digits=4))")
9 end
```

4. Fit the parameters automatically ⇌

Rather than tuning sliders by hand, we let `Optimization.jl` minimize the sum-of-squares loss directly. Because the loss only needs a forward `solve`, we can use a derivative-free or a gradient-based local

optimizer — here we use Optim.jl's Nelder–Mead via OptimizationOptimJL.

sir_loss (generic function with 1 method)

```
1 function sir_loss(p, _)
2     prob = remake(sir_prob; p=p)
3     sol = solve(prob, Tsit5(); saveat=tsteps_sir)
4     sol.retcodes == ReturnCode.Success || return Inf
5     return sum(abs2, sir_data .- Array(sol))
6 end
```

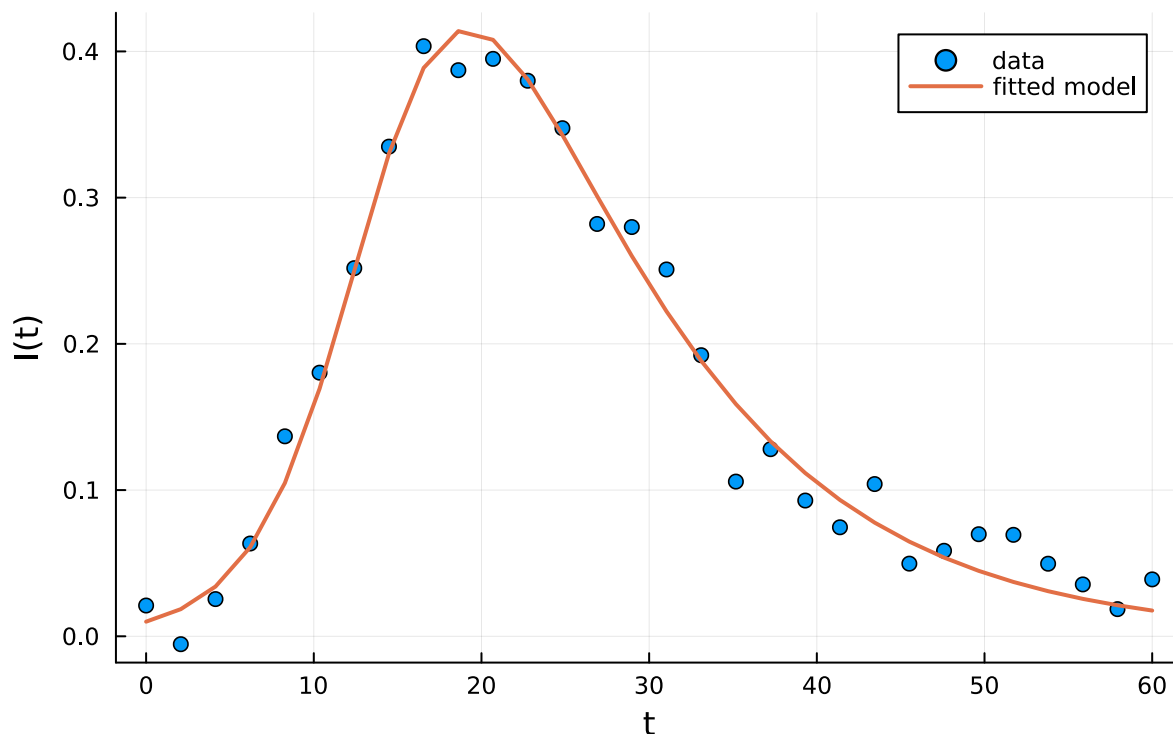
► SciMLBase.OptimizationSolution{Float64, 1, Vector{Float64}, OptimizationBase.OptimizationCache{Op

```
1 begin
2     p0 = [0.2, 0.3] # initial guess, deliberately off from the truth
3     optf4 = OptimizationFunction(sir_loss)
4     optprob4 = OptimizationProblem(optf4, p0)
5     fit_res = Optimization.solve(optprob4, OptimizationOptimJL.NelderMead())
6 end
```

► (fitted_β = 0.403381, fitted_γ = 0.0982194, true_β = 0.4, true_γ = 0.1)

```
1 (fitted_β=fit_res.u[1], fitted_γ=fit_res.u[2], true_β=true_β, true_γ=true_γ)
```

Recovered SIR fit



```
1 begin
2     fitted_sol = solve(remake(sir_prob; p=fit_res.u), Tsit5(); saveat=tsteps_sir)
3     scatter(tsteps_sir, sir_data[2, :]; label="data", xlabel="t", ylabel="I(t)")
4     plot!(tsteps_sir, Array(fitted_sol)[2, :]; label="fitted model", lw=2,
5           title="Recovered SIR fit")
6 end
```

Takeaways ⇄

- Parameter estimation is just optimization: wrap `solve` in a loss function and hand it to `Optimization.jl`.
- `remake(prob; p=...)` avoids rebuilding the `ODEProblem` from scratch on every loss evaluation — a small but important performance detail.
- For harder, higher-dimensional or multimodal problems, swap in a global optimizer (`OptimizationBBO`, `OptimizationCMAEvolutionStrategy`) or a Bayesian approach (`Turing.jl` on top of the same forward model) with no change to the ODE code itself.